Université des Sciences et de la Technologie Houari Boumediene
Faculté d'Informatique

TP ALGO3

Support TP programmation C - les piles et les files -

Fait par :

Nom et prénom : ABDELMALEK BENMEZIANE

Contents

1 Les piles (Stacks) 1

1.1 La définition d'une pile 1

1.2 La représentation d'une pile 1

1.3 Avantages et inconvénients d'une pile 1

1.4 Les applications du pile 2

1.5 Les opérations fondamentales sur les piles 2

1.6 L'implémentation des primitives en langage C 3

1.7 Exemple d'un programme qui manipule les piles 4

1.8 Conclusion 6

2 Les files (Queues) 7

2.1 La définition d'une file 7

2.2 La représentation d'une file 7

2.3 Avantages et inconvénients d'une file 7

2.4 Variantes 8

2.5 Les applications du file 8

2.6 Les opérations fondamentales sur les files 8

2.7 L'implémentation des primitives en langage C 9

2.8 Exemple d'un programme qui manipule les files 11

2.9 Conclusion 12

Chapter 1

Les piles (Stacks)

1.1 La définition d'une pile

Une pile (stack) est une **structure de données linéaire** qui fonctionne selon le principe du **LIFO (Last In, First Out)**. Cela signifie que le dernier élément inséré est le premier à être retiré. Autrement dit, on ajoute et on retire les éléments par un seul bout appelé sommet **(top)**.

1.2 La représentation d'une pile

Une pile peut être représentée à l'aide de deux manières :

- Représentation statique : à travers d'un tableau.
- Représentation chaînée : à travers d'une liste chaînée.

1.3 Avantages et inconvénients d'une pile

Avantages	**Inconvénients**
Simple à implémenter	Taille fixe si tableau
Utilisation logique (LIFO)	Peut devenir inefficace pour de grandes données

Table 1.1: Avantages et inconvénients d'une pile (Stack)

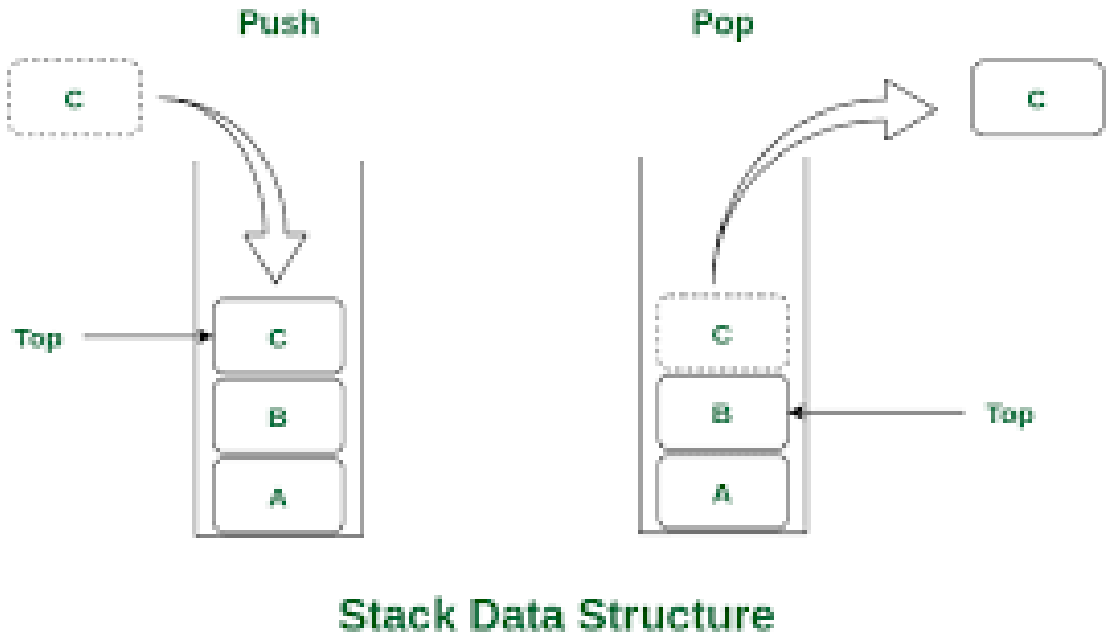


Figure 1.1: Stack

1.4 Les applications du pile

- Gestion de la pile d'appels (fonctions imbriquées).
- Évaluation d'expressions arithmétiques (postfix/prefix).
- Annulation d'opérations (Undo/Redo).
- Parcours en profondeur (DFS) dans les graphes.

1.5 Les opérations fondamentales sur les piles

Les opérations fondamentales (les primitives) des piles suivant la représentation chaînée sont :

- init(stack* s) : initialise une pile.
- is_empty(stack* s) : teste si une pile est vide.
- push(stack* s, int item) : insère un élément au sommet.
- pop(stack* s) : supprime un élément au sommet.

- top(stack* s) : retourne l'élément au sommet de pile.

1.6 L'implémentation des primitives en langage C

Les primitives seront implémentées dans un fichier avec l'extension ".h" pour indique que c'est une bibliothèque par exemple "stack.h".

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node{
    int data;
    struct Node* next;
} Node;

typedef struct stack{
    Node* top;
}stack;


void init(stack* s){
    s->top = NULL;
}

int is_empty(stack* s){
    if(s->top == NULL){
        return 1;
        }else{
                return 0;
        }
}

void push(stack* s, int item){

        Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->data = item;
    newNode->next = s->top;
    s->top = newNode;
}
```

```

int pop(stack* s){

        if (is_empty(s)) {
        return -1;  // or any appropriate value
        // to indicate stack underflow
    }

    Node* poppedNode = s->top;
    int poppedData = poppedNode->data;
    s->top = s->top->next;
    free(poppedNode);

    return poppedData;
}

int top(stack* s){

    if (is_empty(s)) {
        printf("Stack is empty. Cannot peek item.\n");
        return -1;  // or any appropriate value
        // to indicate stack underflow
    }

    return s->top->data;
}
```

1.7 Exemple d'un programme qui manipule les piles

On crée un fichier ".c" (exemple : stack.c) et on inclus le fichier bibliothèque qui contient les primitives des piles.

```
#include <stdio.h>
#include <stdlib.h>
#include "stack.h" // including header file

int main() {

        stack s;    // Declare the stack
```

```c

    init(&s);   // Initialize the stack

    // Push some values onto the stack
    push(&s, 10);
    push(&s, 20);
    push(&s, 30);

    // Peek the top value of the stack
    printf("Top of the stack: %d\n", top(&s));

    // Pop values from the stack
    printf("Popped: %d\n", pop(&s));  // Should pop 30
    printf("Popped: %d\n", pop(&s));  // Should pop 20

    // Check if the stack is empty
    if (is_empty(&s)) {
        printf("The stack is empty.\n");
    } else {
        printf("The stack is not empty.\n");
    }

    // Pop the remaining value
    printf("Popped: %d\n", pop(&s));  // Should pop 10

    // Finally check if the stack is empty again
    if (is_empty(&s)) {
        printf("The stack is empty.\n");
    } else {
        printf("The stack is not empty.\n");
    }


        return 0;
}
```

1.8 Conclusion

En conclusion, la pile constitue une structure de données fondamentale en informatique, caractérisée par son mode de fonctionnement LIFO (Last In, First Out). Sa simplicité d'implémentation et sa logique d'accès séquentiel en font un outil essentiel dans la gestion des appels de fonctions, l'évaluation d'expressions et le contrôle des flux d'exécution.

Malgré certaines limitations, notamment lorsqu'elle est implémentée à l'aide de tableaux de taille fixe, la pile demeure une base indispensable pour la compréhension et la mise en œuvre de structures de données plus complexes. La maîtrise de ses opérations et de son implémentation en langage C permet de mieux appréhender la gestion de la mémoire, les algorithmes récursifs et l'organisation interne des programmes.

Chapter 2

Les files (Queues)

2.1 La définition d'une file

Une file (queue) est une **structure de données linéaire** qui fonctionne selon le principe du **FIFO (First In, First Out)**. Cela signifie que le premier élément inséré est le premier à être retiré. L'insertion se fait à l'arrière **(rear)** et la suppression à l'avant **(front)**.

2.2 La représentation d'une file

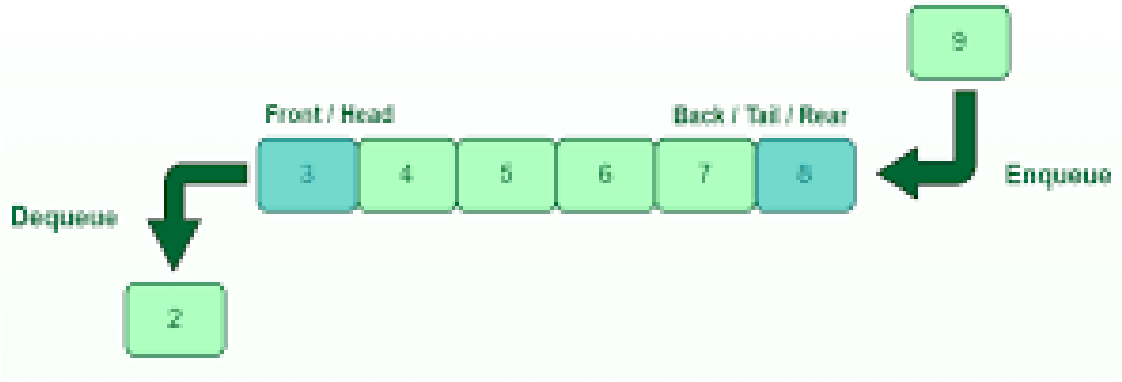
Une file peut être représentée à l'aide de deux manières :

- Représentation statique : à travers d'un tableau avec deux indices.
- Représentation chaînée : à travers d'une liste chaînée.

2.3 Avantages et inconvénients d'une file

Avantages	**Inconvénients**
Gestion efficace de flux de données	Peut être lente si mal implémentée
Facile à comprendre (FIFO)	Taille fixe si tableau simple

Table 2.1: Avantages et inconvénients d'une file (Queue)



Queue Data Structure

Figure 2.1: Queue

2.4 Variantes

- File simple : insertion arrière, suppression avant.
- File circulaire : indices se réinitialisent.
- File à double extrémité (deque) : insertion/suppression possible aux deux extrémités.

2.5 Les applications du file

- Gestion de tâches (CPU scheduling, files d'attente).
- Buffers d'entrée/sortie (clavier, imprimante, réseau).
- Algorithme de parcours en largeur (BFS).

2.6 Les opérations fondamentales sur les files

Les opérations fondamentales (les primitives) des files suivant la représentation chaînée sont :

- init(queue* q) : initialise une file.

- isEmpty(queue* q) : teste si une file est vide.
- enqueue(queue* q, int value) : insère un élément en arrière du file.
- dequeue(queue* q) : supprime un élément au début du file.
- getFront(queue* q) : retourne l'élément au début du file sans le supprimer.

2.7 L'implémentation des primitives en langage C

Les primitives seront implémentées dans un fichier avec l'extension ".h" pour indique que c'est une bibliothèque par exemple "queue.h".

```
#include <stdio.h>
#include <stdlib.h>

// Structure for a node in the linked list
typedef struct Node {
    int data;
    struct Node* next;
}Node;

// Structure for the queue
typedef struct queue{
    Node* front;
    Node* rear;
}queue;

void init(queue* q) {
    q->front = NULL;
    q->rear = NULL;
}

int isEmpty(queue* q) {
        if(q->front==NULL){
                return 1;
        }else{
                return 0;
        }
```

```
}

void enqueue(queue* q, int value) {

    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->data = value;
    newNode->next = NULL;

    if (isEmpty(q)) {
        q->front = q->rear = newNode;
    } else {
        q->rear->next = newNode;
        q->rear = newNode;
    }
}

int dequeue(queue* q) {

    if (isEmpty(q)) {
        printf("Queue is empty\n");
        return -1; // Return a sentinel value indicating failure
    }

    Node* temp = q->front;
    int dequeuedValue = temp->data;

    q->front = q->front->next;
    free(temp);

    if (q->front == NULL) {
        q->rear = NULL;
    }

    return dequeuedValue;
}

int getFront(queue* q){
        return q->front->data;
}
```

2.8 Exemple d'un programme qui manipule les files

On crée un fichier ".c" (exemple : queue.c) et on inclus le fichier bibliothèque qui contient les primitives des files.

```c
#include <stdio.h>
#include <stdlib.h>
#include "queue.h" // including the header file


int main() {

    queue q;
    init(&q);

    // Enqueue some values
    enqueue(&q, 10);
    enqueue(&q, 20);
    enqueue(&q, 30);

    // Check the first element
    printf("First element in the queue: %d\n", getFront(&q));

    // Dequeue elements
    printf("Dequeued element: %d\n", dequeue(&q));
    printf("Dequeued element: %d\n", dequeue(&q));

    // Check the first element after dequeue
    printf("First element in the queue after dequeues: %d\n", getFront(&q));

    // Dequeue the last element
    printf("Dequeued element: %d\n", dequeue(&q));

    // Try to dequeue from an empty queue
    printf("Dequeued element: %d\n", dequeue(&q));

    return 0;
}
```

2.9 Conclusion

En conclusion, la file est une structure de données linéaire tout aussi essentielle que la pile. Son principe de fonctionnement FIFO (First In, First Out) permet une gestion ordonnée et équitable des éléments, rendant cette structure particulièrement adaptée aux systèmes où les opérations doivent être traitées dans leur ordre d'arrivée, comme les files d'attente, les buffers ou la planification de tâches.

Grâce à sa simplicité conceptuelle et à ses nombreuses variantes (file circulaire, double file, etc.), la file constitue un outil fondamental pour la conception d'algorithmes efficaces et la gestion de flux de données continus. Sa maîtrise en langage C offre une compréhension approfondie des mécanismes de gestion de la mémoire et des processus séquentiels, ouvrant la voie vers l'étude de structures plus avancées telles que les graphes ou les systèmes multitâches.